
PART 3 OF 5. THE WEEK THAT DECIDES WHETHER ANYBODY TRUSTS THIS.

Run it by hand before you send anything

You have a rule and a query that returns a list you believe. The temptation now is to wire it up to send messages, because that is the part that looks like finishing.

Do not do that yet. Run it by hand for a week first.

I know how that sounds. You automated something so you would stop doing it manually, and step one is to do it manually. But this is the week where you find out that your tracker is wrong, that two people share a name, that somebody's items all belong to a manager who left, and that the thing you were about to send to fifteen people would have been wrong for four of them.

This part is also where the code gets written, and where I explain how somebody who does not write software for a living gets software written anyway.

1 Run the query every morning and read every row

Not the count. The rows. Give it ten minutes with coffee.

You are looking for three things. Rows that are wrong. Rows that are right but would be rude to send. Rows that repeat every single day and never change, because those are a different problem than lateness.

Keep the output somewhere you can compare against yesterday. A file per day is enough. What you want to see by Friday is whether the list is stable, and whether the things on it are the things you would actually have chased.

EXAMPLE

My Monday list had nineteen items. By Wednesday I had worked out that six of them belonged to one person who had changed teams, and the tool would have messaged them every day about work that was no longer theirs.

WHY THIS MATTERS: this week is not about the data being clean. It is about you knowing exactly how it is dirty, before anybody else sees the output.

2 Get the code written without writing it yourself

I have a computer engineering degree and I have not written production code in years. I did not write most of this by hand. I described what I wanted to a code assistant, in the same words I would use with a colleague, and corrected it until it was right. I used Claude Code.

The skill is not typing. It is being precise about what you want, which is exactly what parts 1 and 2 were for. Somebody who has not done that work asks for a status bot and gets something generic. You can ask for something specific because you already wrote the rule down.

EXAMPLE

Here is our API documentation. Write a script that pulls every open item with an owner, a state and a target date. Print two lists. One for items past their target date and not closed. One for items due in the next seven days that have not changed state in fourteen days. Do not send anything, just print. Include the owner, the title, the state and the number of days late.

WHY THIS MATTERS: the barrier to building small tools stopped being programming ability. It is now knowing precisely what you want, and being willing to keep correcting until you get it.

3 Make it print instead of send, and keep it that way for a week

Whatever you build, the first version should end in a print statement, not a message. That one decision is what makes this safe to iterate on.

Add a real send later, behind something you have to switch on deliberately. Not a setting you might forget. Something you have to type.

EXAMPLE

My first version wrote a file with the exact message it would have sent to each person. I read that file every morning for a week. Nobody received anything, and I found four wrong messages I would otherwise have sent.

WHY THIS MATTERS: the first message anybody receives from this sets whether they ever take it seriously. You cannot undo it, so buy yourself a week where mistakes are free.

4 Fix the data problems the week exposes, and do it out loud

Everything the week surfaced is a real finding about how your team works. Items with no dates. Owners who left. A state everybody uses that means something different to each of them.

Take those to your team as findings, not as complaints, and fix them before you automate on top of them. This is the most valuable side effect of the whole project and it happens before the tool exists.

EXAMPLE

I brought a list to a team meeting. Forty percent of our open items had no target date. Nobody had ever been asked for one, so nobody had ever set one. That conversation was worth more than the script.

WHY THIS MATTERS: automating on top of bad data does not surface the bad data, it launders it. Now it is a report and people believe it.

5 Decide what a good day looks like before you go live

Write down what you expect to see when this is working. How many messages a day is normal. How many is too many. What number would make you switch it off.

Then log every run. Not because you will read the logs, but because the day something goes strange you will want to know whether it started today or three weeks ago.

EXAMPLE

Mine was under ten messages on a normal day. More than twenty means the data broke or something genuinely went wrong with the program, and either way I want to look before anybody gets messaged.

WHY THIS MATTERS: a tool that quietly triples its output is how people learn to ignore it. Knowing your normal is what lets you catch that on the first day.

A week of reading a list you did not send is the least exciting part of this and it is the part that decides whether anybody trusts the result.

By the end of it you will have a script that prints something correct, a short list of things about your team's data you did not know on Monday, and a number in your head for what normal looks like.

Part 4 is next: how the message is written and where it gets sent, which matters more than everything you have built so far.